

ExpensePro

Income & Expense Manager

Personal Finance Management Progressive Web App

PHP 8+ PDO MySQL (InnoDB, utf8mb4) Vanilla JavaScript Apache/XAMPP

Rohit Vadher BCA Sem-5

Version 1.0 2026

Contents

1 Project Abstract	2
2 Project Objectives	2
3 Problem Statement	2
4 System Overview	2
5 Functional Scope	2
6 Technology Stack	3
7 System Architecture	3
8 Folder Architecture	3
9 Frontend Architecture	4
10 Backend Architecture	4
11 API Architecture	4
12 Database Architecture	4
12.1 Database Tables	4
12.1.1 budgets — Spending limits per period	4
12.1.2 categories — Per-user income/expense groups	5
12.1.3 csv_imports — Import history and error logs	5
12.1.4 login_attempts — Brute-force guard	5
12.1.5 notifications — Budget and login alerts	6
12.1.6 quick_templates — One-tap entry presets	6
12.1.7 rate_limits — Generic API buckets	6
12.1.8 transactions — Every income/expense entry	7
12.1.9 users — Registered accounts	7
13 Authentication Workflow	8
14 User Workflow	8
15 Transaction Workflow	9
16 Category Workflow	9
17 Budget Workflow	9
18 Analytics Workflow	9
19 Report Workflow	10
20 Import Workflow	10
21 Export Workflow	10
22 Notification Workflow	11

23 Profile Workflow	11
24 Validation Workflow	11
25 Security Workflow	12
26 Financial Calculation Workflow	12
27 Error Handling Architecture	13
28 Asset Loading Architecture	13
29 CDN/Lazy Loading Architecture	13
30 PWA Architecture	13
31 Performance Strategy	13
32 Responsive Design	13
33 UI/UX Design System	13
34 API Endpoint Reference	14
35 API Request Flow	14
36 Security Controls	14
37 Data Flow	14
38 Request Lifecycle	14
39 User Journey	15
40 Deployment Architecture	15
41 XAMPP Installation	15
42 MySQL Installation/Setup	15
43 Configuration	15
44 Browser Compatibility	15
45 Project Statistics	15
46 Screenshots	17
47 Project Structure	25
48 Limitations	26
49 Maintenance Notes	26
50 Conclusion	26
51 Student/Project Information	26

Project Abstract

ExpensePro is a PHP and MySQL based personal finance management Progressive Web App designed to manage income, expenses, categories, budgets, analytics, reports, notifications and financial data import/export through a modular web architecture. Guests start on a public landing page; registered users sign in to a dashboard that summarizes the current month and links every feature. The system runs on Apache/XAMPP with PDO access to a MySQL (InnoDB, utf8mb4) database, needs no build step, installs as a PWA and degrades gracefully offline.

Project Objectives

- Record income and expense entries in seconds with validation at every step.
- Organize spending with private income/expense categories.
- Stay within weekly, monthly or yearly budgets with live progress.
- Visualize twelve-month trends and category breakdowns.
- Produce CSV and PDF records filtered exactly like the on-screen reports.
- Import history from CSV with per-row validation feedback.
- Keep every byte of one user's data invisible to every other user.

Problem Statement

Household spending spreads across cash, cards and apps, so monthly totals surprise their owners. Spreadsheets demand discipline and offer no budgets, alerts or mobile entry. ExpensePro answers with a private, mobile-first ledger: fast entry, automatic totals, budget guardrails and one-click exports, all self-hosted on XAMPP with MySQL.

System Overview

A front controller routes whitelisted pages to server-rendered shells. Page controllers call a single JSON API client; endpoints enforce authentication, CSRF, validation and ownership, then delegate to services and repositories. MySQL stores exact decimals; one finance module computes every total shown anywhere.

Functional Scope

Registration, login, logout, remember-me sessions, dashboard, transactions (CRUD, search, filters, pagination), categories, budgets, analytics, reports, CSV import/export, PDF/print export, notifications, quick templates, profile, password change, command palette, mobile navigation, help, offline page and PWA install/update flows.

Technology Stack

Layer	Technology
Backend	PHP 8+
Database	MySQL
Database Engine	InnoDB
Database Access	PDO
Frontend	Vanilla JavaScript
CSS	Tailwind CSS
Charts	ApexCharts 3.44.0 (lazy-loaded)
Icons	Lucide 1.47.0 where used, local SVG set
Date Tools	Flatpickr 4.6.13 pickers, Day.js 1.11.23 math (lazy-loaded)
PDF	html2pdf.js 0.10.2 client engine plus server printable report
PWA	Service Worker + Web App Manifest
Server	Apache/XAMPP

System Architecture

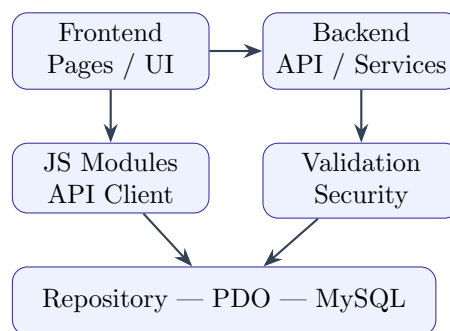


Figure 1: High-level architecture: UI and services meet at repositories over PDO/MySQL.

Folder Architecture

ExpensePro/	
api/	14 JSON endpoints (HTTP boundary only)
assets/	css (core/layout/components/features/pages), js (core/services/components/features/pages/vendor), icons
backend/	18 modules: core, validation, http, security, domain, support, calculations, database, services
database/	schema.sql + idempotent migrations
docs/	final documentation, screenshots, diagrams
includes/	config, PDO, auth, API guards, finance
layouts/	guest/app headers, sidebar, mobile nav, footer
offline/	self-contained offline page
pages/	14 server-rendered pages
index.php	front controller (?page= routing)
manifest.json	sw.js PWA shell

Frontend Architecture

Server-rendered shells plus page controllers sharing one core: single API client (one JSON parser, CSRF header, AbortController, network-vs-server errors), central error handler (401/403/404/409/422/429/500 mapping, field errors), one validation layer, one finance preview module and one button/form state machine. One modal controller, one toast system, one SRI-pinned lazy vendor loader. No inline handlers; a delegated dispatcher routes data-ep-action clicks.

Backend Architecture

Controllers handle method, auth, CSRF, validation and responses only. Services hold workflows (transactions, categories); repositories hold SQL (PDO prepared statements, user-scoped). Focused modules cover escaping, validation, money math, CSRF, rate limiting, responses, notifications and budget periods.

API Architecture

Uniform envelope `{success, message, data, errors}`. Guards: `requireApiAuth` (401), `requireApiCsrftoken` (403), `requireJsonBody` (400), `requireApiMethod` (405), `requireOwnedCategory` (404/422). Duplicates 409, rate limits 429, server faults safe 500. Downloads use correct content types and filenames.

Database Architecture

MySQL 8+, InnoDB, utf8mb4. Money is exact `DECIMAL(12,2)`. Every user-owned row carries `user_id` with foreign keys cascading from `users`; transaction categories are delete-restricted; budget and template categories null out safely. Ownership plus performance indexes cover auth, filtering, reporting and cleanup.

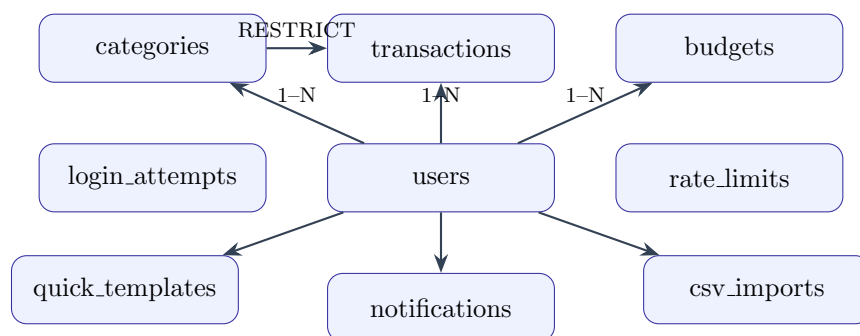


Figure 2: Entity relationships. All user data cascades from `users`; categories protect live transactions.

Database Tables

12.1.1 budgets — Spending limits per period

Table	Column	Type	Null	Key	Default
budgets	id	int(11)	NO	PRI	NULL
budgets	user_id	int(11)	NO	MUL	NULL

Table	Column	Type	Null	Key	Default
budgets	category_id	int(11)	YES	MUL	NULL
budgets	amount	decimal(12,2)	NO	-	NULL
budgets	period	enum('weekly','monthly','yearly')	YES	-	monthly
budgets	start_date	date	NO	-	NULL
budgets	end_date	date	NO	-	NULL
budgets	created_at	timestamp	NO	-	current_timestamp()
budgets	updated_at	timestamp	NO	-	current_timestamp()

Indexes: PRIMARY(id), category_id(category_id), idx_user_period(user_id+start_date+end_date), idx_user_period_category(user_id+period+category_id), idx_user_created(user_id+created_at). Foreign keys: user_id references users(id) ON DELETE CASCADE; category_id references categories(id) ON DELETE SET.

12.1.2 categories — Per-user income/expense groups

Table	Column	Type	Null	Key	Default
categories	id	int(11)	NO	PRI	NULL
categories	user_id	int(11)	NO	MUL	NULL
categories	name	varchar(100)	NO	-	NULL
categories	type	enum('income','expense')	NO	-	NULL
categories	icon	varchar(50)	NO	-	NULL
categories	color	varchar(7)	NO	-	NULL
categories	created_at	timestamp	NO	MUL	current_timestamp()

Indexes: PRIMARY(id), uq_user_name_type(user_id+name+type), idx_user_type(user_id+type), idx_created_at(created_at). Foreign keys: user_id references users(id) ON DELETE CASCADE.

12.1.3 csv_imports — Import history and error logs

Table	Column	Type	Null	Key	Default
csv_imports	id	int(11)	NO	PRI	NULL
csv_imports	user_id	int(11)	NO	MUL	NULL
csv_imports	filename	varchar(255)	NO	-	NULL
csv_imports	row_count	int(11)	YES	-	0
csv_imports	success_count	int(11)	YES	-	0
csv_imports	error_count	int(11)	YES	-	0
csv_imports	errors_log	longtext	YES	-	NULL
csv_imports	status	enum('completed','partial','failed')	YES	-	completed
csv_imports	created_at	timestamp	NO	-	current_timestamp()

Indexes: PRIMARY(id), idx_user_imports(user_id+created_at). Foreign keys: user_id references users(id) ON DELETE CASCADE.

12.1.4 login_attempts — Brute-force guard

Table	Column	Type	Null	Key	Default
login_attempts	id	int(11)	NO	PRI	NULL
login_attempts	email	varchar(255)	NO	MUL	NULL
login_attempts	ip	varchar(45)	NO	-	NULL
login_attempts	attempted_at	timestamp	NO	MUL	current_timestamp()

Indexes: PRIMARY(id), idx_email_ip_time(email+ip+attempted_at), idx_attempted_at(attempted_at).

12.1.5 notifications — Budget and login alerts

Table	Column	Type	Null	Key	Default
notifications	id	int(11)	NO	PRI	NULL
notifications	user_id	int(11)	NO	MUL	NULL
notifications	type	enum('over_budget','info','login')	NO	-	info
notifications	title	varchar(200)	NO	-	NULL
notifications	message	text	NO	-	NULL
notifications	link	varchar(255)	YES	-	NULL
notifications	is_read	tinyint(1)	YES	-	0
notifications	created_at	timestamp	NO	-	current_timestamp()

Indexes: PRIMARY(id), idx_user_read(user_id+is_read+created_at), idx_user_type_title_created(user_id+type+title+created_at)
Foreign keys: user_id references users(id) ON DELETE CASCADE.

12.1.6 quick_templates — One-tap entry presets

Table	Column	Type	Null	Key	Default
quick_templates	id	int(11)	NO	PRI	NULL
quick_templates	user_id	int(11)	NO	MUL	NULL
quick_templates	name	varchar(100)	NO	-	NULL
quick_templates	category_id	int(11)	YES	MUL	NULL
quick_templates	type	enum('income','expense')	NO	-	NULL
quick_templates	amount	decimal(12,2)	YES	-	NULL
quick_templates	description	text	YES	-	NULL
quick_templates	usage_count	int(11)	YES	-	0
quick_templates	created_at	timestamp	NO	MUL	current_timestamp()
quick_templates	updated_at	timestamp	NO	-	current_timestamp()

Indexes: PRIMARY(id), category_id(category_id), idx_user_usage(user_id+usage_count), idx_created_at(created_at)
Foreign keys: user_id references users(id) ON DELETE CASCADE; category_id references categories(id) ON DELETE SET.

12.1.7 rate_limits — Generic API buckets

Table	Column	Type	Null	Key	Default
rate_limits	id	int(11)	NO	PRI	NULL
rate_limits	rate_key	varchar(191)	NO	MUL	NULL
rate_limits	ip	varchar(45)	NO	-	NULL

Table	Column	Type	Null	Key	Default
rate_limits	attempted_at	timestamp	NO	MUL	current_timestamp()

Indexes: PRIMARY(id), idx_key_time(rate_key+attempted_at), idx_attempted_at(attempted_at).

12.1.8 transactions — Every income/expense entry

Table	Column	Type	Null	Key	Default
transactions	id	int(11)	NO	PRI	NULL
transactions	user_id	int(11)	NO	MUL	NULL
transactions	category_id	int(11)	NO	MUL	NULL
transactions	type	enum('income','expense')	NO	-	NULL
transactions	amount	decimal(12,2)	NO	-	NULL
transactions	description	text	YES	-	NULL
transactions	date	date	NO	MUL	NULL
transactions	created_at	timestamp	NO	-	current_timestamp()
transactions	updated_at	timestamp	NO	-	current_timestamp()

Indexes: PRIMARY(id), category_id(category_id), idx_user_date(user_id+date), idx_user_type_date(user_id+type), idx_user_category(user_id+category_id), idx_date(date). Foreign keys: user_id references users(id) ON DELETE CASCADE; category_id references categories(id).

12.1.9 users — Registered accounts

Table	Column	Type	Null	Key	Default
users	id	int(11)	NO	PRI	NULL
users	name	varchar(100)	NO	-	NULL
users	email	varchar(255)	NO	UNI	NULL
users	password	varchar(255)	NO	-	NULL
users	role	enum('user','admin')	YES	-	user
users	currency	varchar(10)	YES	-	INR
users	monthly_budget	decimal(12,2)	YES	-	NULL
users	remember_token	varchar(255)	YES	MUL	NULL
users	created_at	timestamp	NO	MUL	current_timestamp()
users	updated_at	timestamp	NO	-	current_timestamp()

Indexes: PRIMARY(id), email(email), idx_created_at(created_at), idx_remember_token(remember_token).

Authentication Workflow

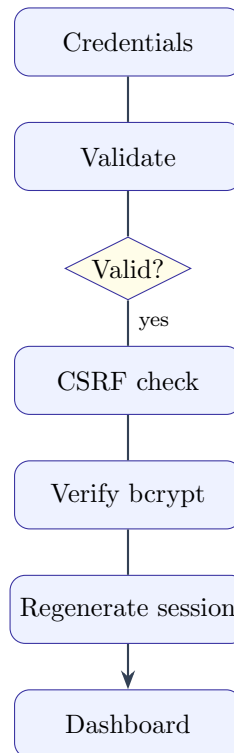


Figure 3: Login flow. Failures return 401/422 without revealing which field failed remotely.

Registration validates input, rejects duplicate emails (409), inserts the user plus default categories in one transaction and signs in. Logout revokes remember-me tokens and destroys the session. Remember-me tokens are SHA-256 stored, HttpOnly, SameSite=Lax, 30-day, rotated per use. Password change requires the current password and revokes remembered devices.

User Workflow

Guest landing, register or login, dashboard overview, daily entry via transactions or templates, budgets guardrails, analytics and reports review, CSV/PDF export, profile and notification upkeep, logout.

Transaction Workflow

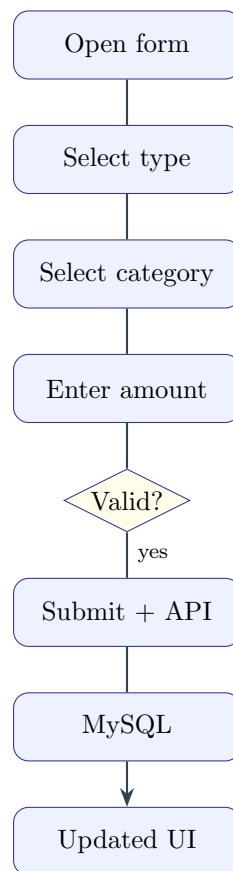


Figure 4: Transaction lifecycle with ownership and duplicate checks server-side.

Category Workflow

Create with name, type, icon and color; duplicates per user, name and type rejected (409); edit keeps type; delete with live transactions either reassigns them to a same-type category or is blocked with guidance.

Budget Workflow

Overall or per-category budgets over weekly, monthly or yearly auto-ranged periods; spent totals, remaining amounts, percentages and over-budget flags computed by the shared finance module; duplicates per category and period rejected.

Analytics Workflow

Twelve-month income, expense and balance series plus category breakdown load through one cancellable request; refresh reuses the pipeline with stale-response guards; charts render only when data exists.

Report Workflow

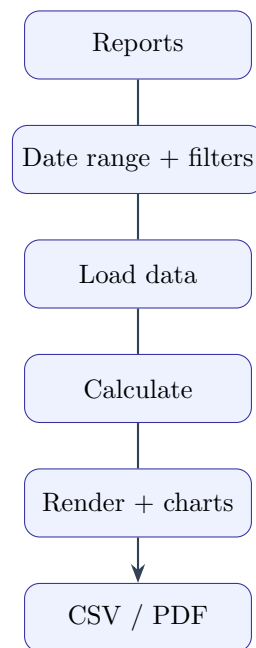


Figure 5: Report pipeline reusing page filters for server-authoritative exports.

Import Workflow

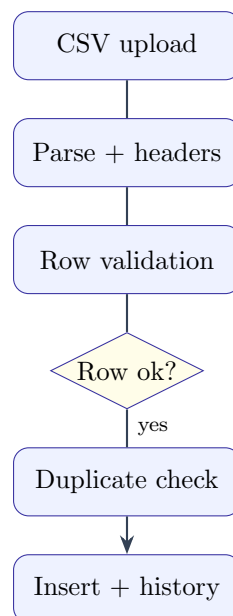


Figure 6: CSV import: 5 MB cap, MIME sniffing, 5000-row cap, per-row errors, atomic history.

Export Workflow

CSV streams with BOM and correct filename; the printable report renders server-side tables for print/PDF. Both reuse the active page filters.

Notification Workflow

Over-budget checks regenerate at most every 30 minutes with same-day deduplication; login notices deduplicate within 15 minutes; only the newest 30 per user are kept; mark-one and mark-all are ownership-scoped.

Profile Workflow

Edits require the current password; email uniqueness is enforced; password change verifies the current password, validates strength, revokes remembered devices, rotates session and CSRF, and returns the fresh token so the next save succeeds.

Validation Workflow

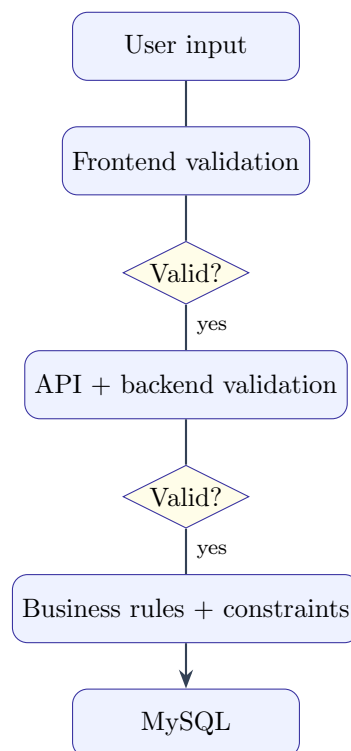


Figure 7: Two-layer validation: instant field errors, then structured 422 API errors.

Rules: email format; password 8+ characters with letter and number; amounts finite, positive, capped with two decimals; YYYY-MM-DD dates; positive ids; enums; hex colors; icon allowlists; names 2–100 chars; category/type match; duplicates rejected.

Security Workflow

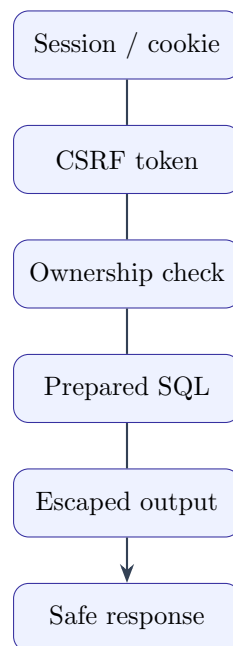
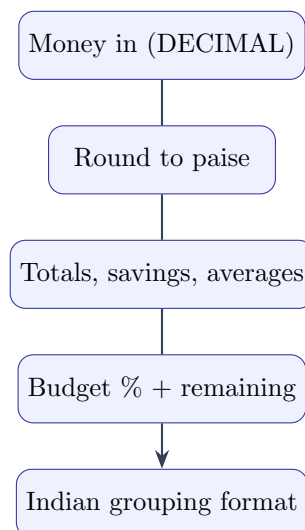


Figure 8: Defense in depth on every protected request.

Controls: bcrypt with rehash, session regeneration and 24-hour idle expiry, HttpOnly SameSite=Lax cookies, 30-minute CSRF rotation with constant-time compare, per-user query scoping, prepared statements, output escaping, allowlisted links/icons, login/register/import rate limits with self-cleaning buckets, generic error responses, CSP/nosniff/framing/referrer headers and denied direct access to code, data and log paths.

Financial Calculation Workflow



Balance = Income – Expense. Savings rate = (Income – Expense)/Income × 100. Remaining = max(0, Limit – Spent); over-budget when Spent > Limit. One module serves dashboard, analytics, reports, budgets, exports and profile.

Error Handling Architecture

Global JSON handlers convert throwables to safe messages; API clients classify aborts, network faults, 401/403/404/409/422/429/500 and invalid JSON distinctly; loaders always resolve to content, empty, error-with-retry or offline states — never stuck spinners.

Asset Loading Architecture

Core scripts plus one page controller per navigation; charts, PDF engine, icons, pickers and date math lazy-load once with timeouts; AbortController plus sequence numbers drop stale responses; event delegation keeps listeners constant.

CDN/Lazy Loading Architecture

Single registry with pinned versions, integrity hashes and dependency order; promise caching prevents duplicates; failures clean up and degrade to native controls or friendly errors. jsDelivr only.

PWA Architecture

Manifest plus versioned service worker: precached icons/offline page, network-only APIs with offline JSON, network-first navigation with offline fallback, stale-while-revalidate statics, old-cache purge, skip-waiting updates with banner, deferred install prompt and version polling.

Performance Strategy

No framework, page-scoped modules, lazy vendors, cancellable requests, indexed paginated queries, immutable versioned-asset caching, gzip output and service-worker caching.

Responsive Design

Desktop sidebar plus top bar from 768px; mobile header, bottom navigation with central action, bottom sheets and filter panels below; safe-area padding; no page-level horizontal overflow at 390–412px widths.

UI/UX Design System

Light theme, system typography, white cards, indigo primary actions, rose danger, emerald success, amber warnings; deterministic button states (disabled while invalid or submitting); modals with focus trap and restore; toasts plus persistent notifications; empty, loading, error and validation states on every data screen.

API Endpoint Reference

Endpoint	Methods	Auth	Purpose
api/login.php	POST	public + CSRF	Sign in, remember-me, login
api/register.php	POST	public + CSRF	Create account + default ca
api/logout.php	POST	session + CSRF	Revoke tokens, destroy sessi
api/version.php	GET	public	Build check for update ban
api/dashboard.php	GET	session	Month KPIs, recent, trend,
api/transactions.php	GET/POST/PUT/DELETE	session (+CSRF write)	Full CRUD, filters, paginati
api/categories.php	GET/POST/PUT/DELETE	session (+CSRF write)	CRUD, safe delete
api/budgets.php	GET/POST/PUT/DELETE	session (+CSRF write)	Period budgets, progress
api/reports.php	GET	session	Summary, categories, trend,
api/export.php	GET	session	CSV download / printable r
api/import.php	GET/POST	session (+CSRF write)	CSV upload, preview data,
api/profile.php	GET/PUT	session (+CSRF write)	Profile, password change
api/quick-templates.php	GET/POST/PUT/DELETE	session (+CSRF write)	Entry presets
api/notifications.php	GET/POST	session (+CSRF write)	Alerts, mark read

API Request Flow

Browser, frontend API client, HTTP request, endpoint, authentication, CSRF, validation, service, repository, MySQL, repository result, service, API response, frontend state, UI — with field errors mapped back to the sending form.

Security Controls

Sessions, cookies, CSRF, authentication, authorization, ownership checks, prepared statements, input validation, output escaping, password hashing, rate limiting, secure responses, HTTP security headers, direct-file protection and safe error handling — each verified in the implementation.

Data Flow

UI state feeds validated requests; services coordinate repositories; MySQL persists decimals and relations; responses refresh exactly the state that changed; stale responses can never overwrite newer ones.

Request Lifecycle

Route whitelist, layout shell, page controller bootstrap, listeners, initial fetch with loading state, render (content/empty/error), user actions with button states, cancellable refetch, modal focus management, cleanup on navigation.

User Journey

Landing, register, dashboard, first categories, first entries or CSV import, budgets, analytics review, report export, profile upkeep. Every step works with keyboard and on a 390px phone.

Deployment Architecture

XAMPP Apache serving the folder, PHP 8 with pdo-mysql/mbstring/fileinfo, MySQL 8 with the imported schema, environment overrides for credentials and base URL, versioned assets for cache-safe releases.

XAMPP Installation

Install XAMPP with PHP 8+, MySQL and Apache; start both services; enable mod-rewrite and mod-headers.

MySQL Installation/Setup

Create database `expensepro (utf8mb4_unicode_ci)`; import `database/schema.sql`; apply `database/migrations` on upgrades; set `EXPENSEPRO_DB_*` credentials.

Configuration

Base URL auto-detect with override; app name/version; Asia/Kolkata timezone; 24-hour sessions; 30-minute CSRF rotation; production-safe errors (nothing displayed, nothing written to files).

Browser Compatibility

Chrome/Edge (full PWA), Firefox (no install prompt), Safari iOS (Add to Home Screen); graceful degradation for AbortController, fetch, dynamic viewport units and backdrop filters.

Project Statistics

Metric	Count (measured)
Total source files	127
PHP files	59
JavaScript modules	24
CSS modules	26
API endpoints	14
Pages	14
Backend modules	18
Database tables	9
Major features	16

Source: analysis of the ExpensePro project repository.

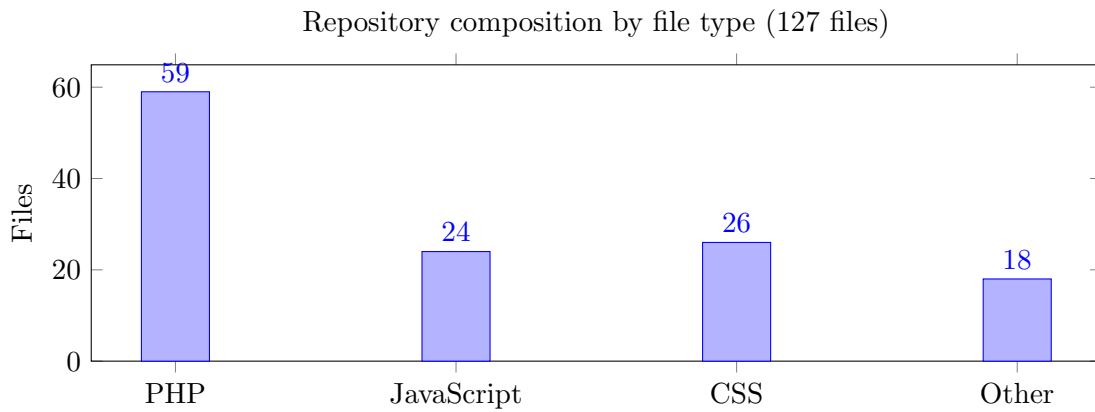


Figure 9: File counts measured from the repository. Source: repository analysis.

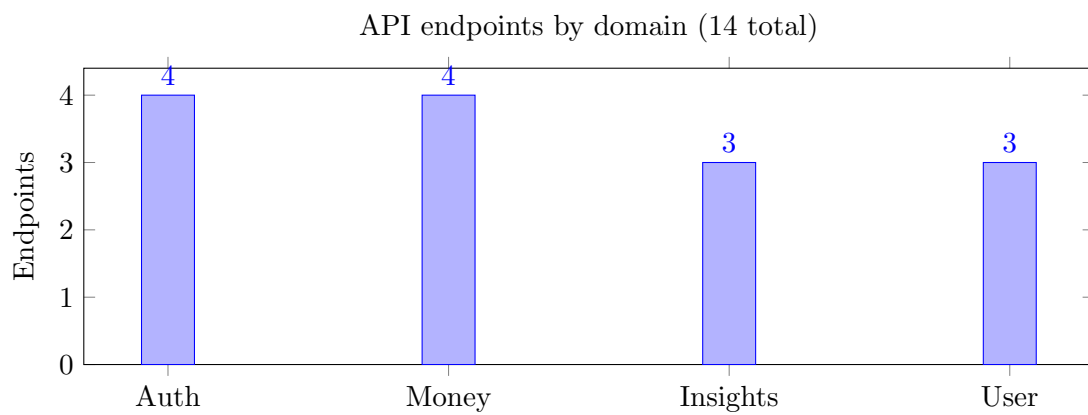


Figure 10: Endpoint domains: auth (login/register/logout/version), money (dashboard/transactions/categories/budgets), insights (reports/export/import), user (profile/notifications/quick-templates).

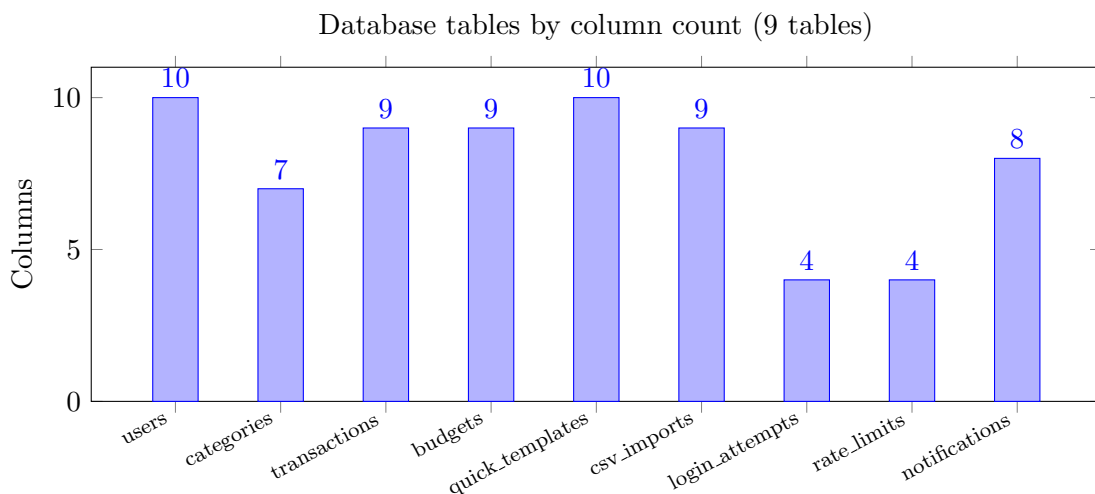


Figure 11: Column counts per table, read from the live MySQL schema.

Screenshots

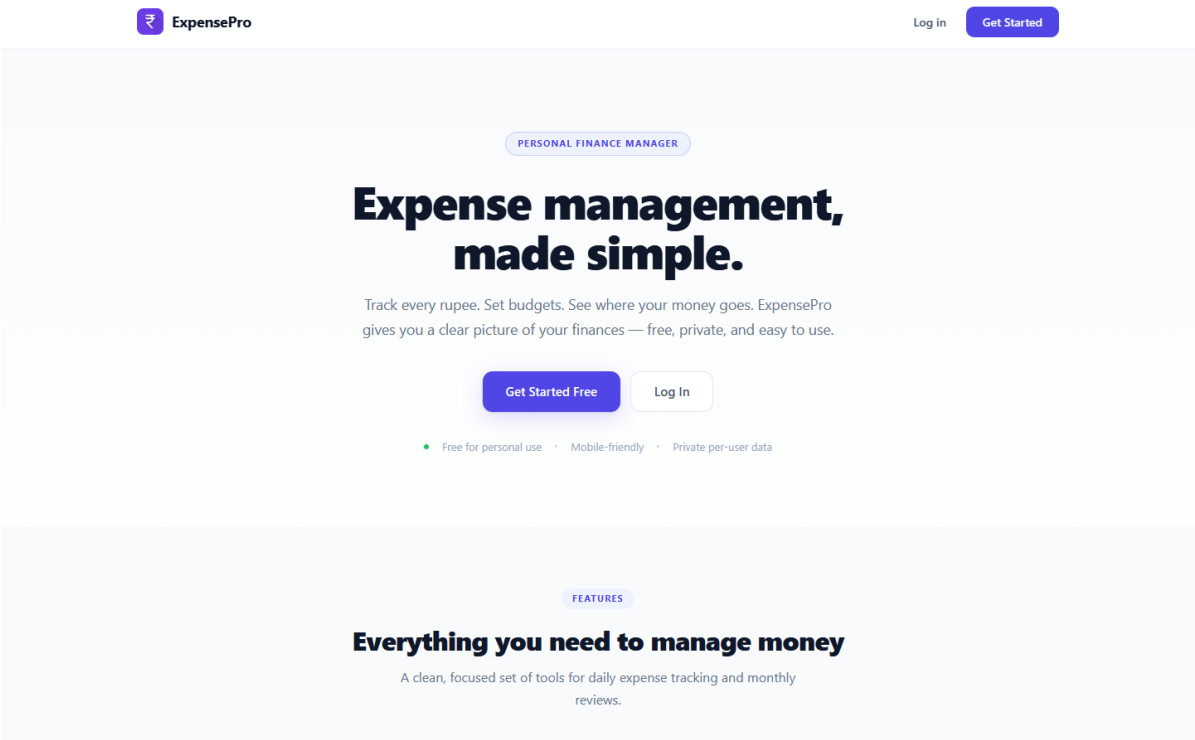


Figure 12: Figure — ExpensePro Landing page.

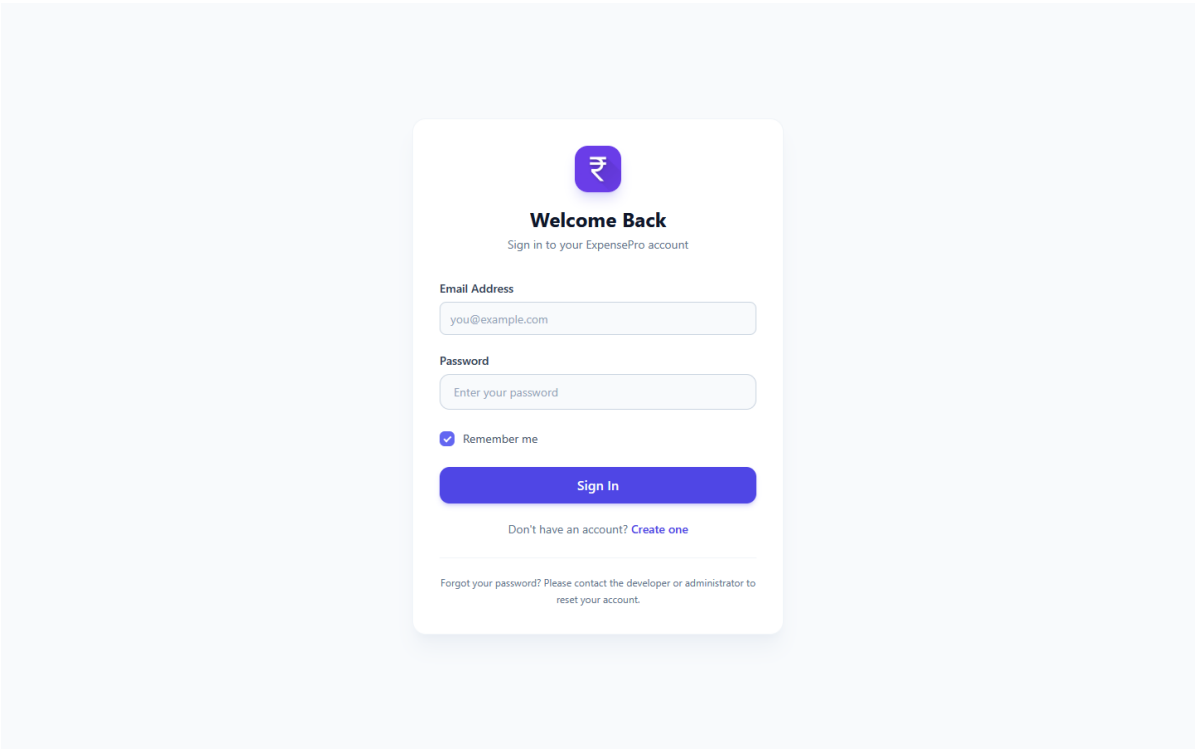


Figure 13: Figure — ExpensePro Login.

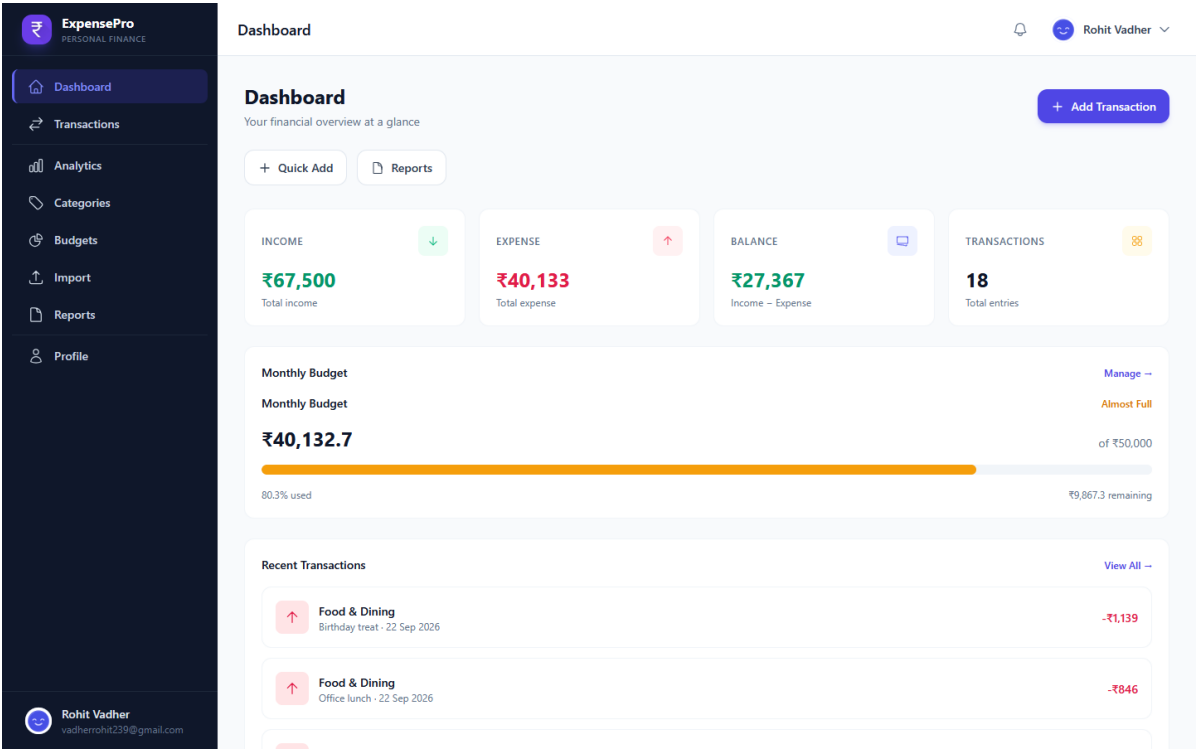


Figure 14: Figure — ExpensePro Dashboard with monthly KPIs.

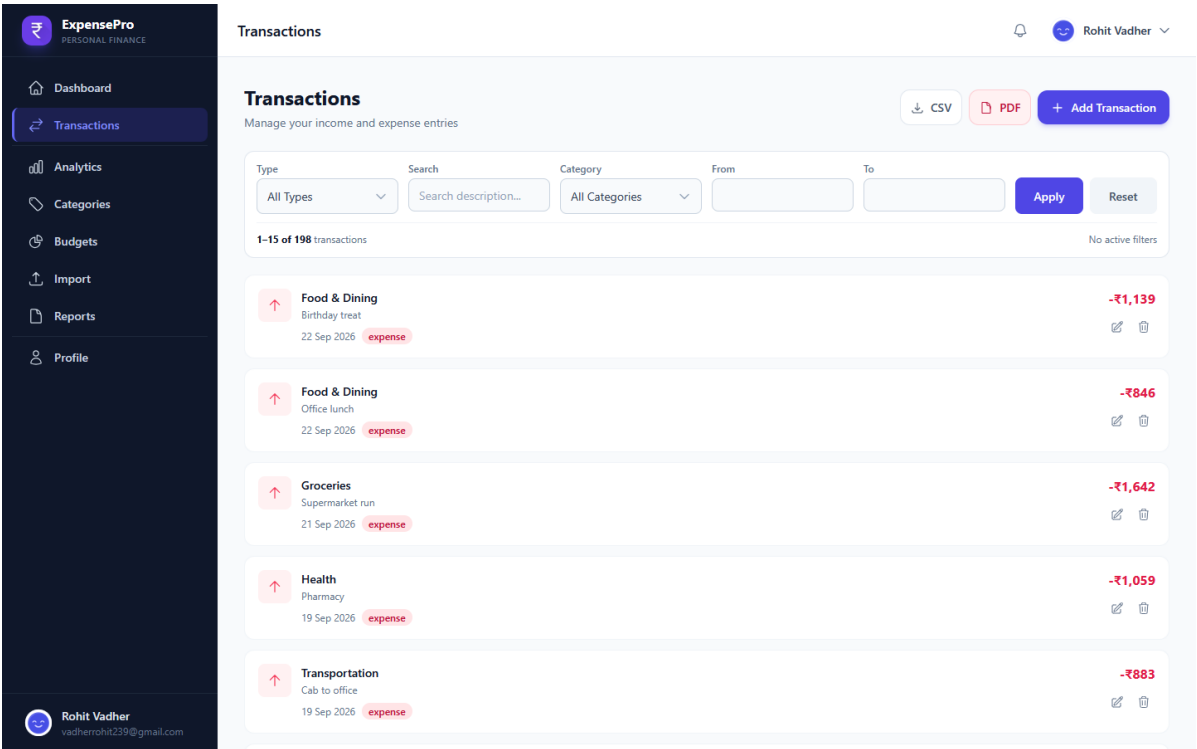


Figure 15: Figure — ExpensePro Transactions with filters.

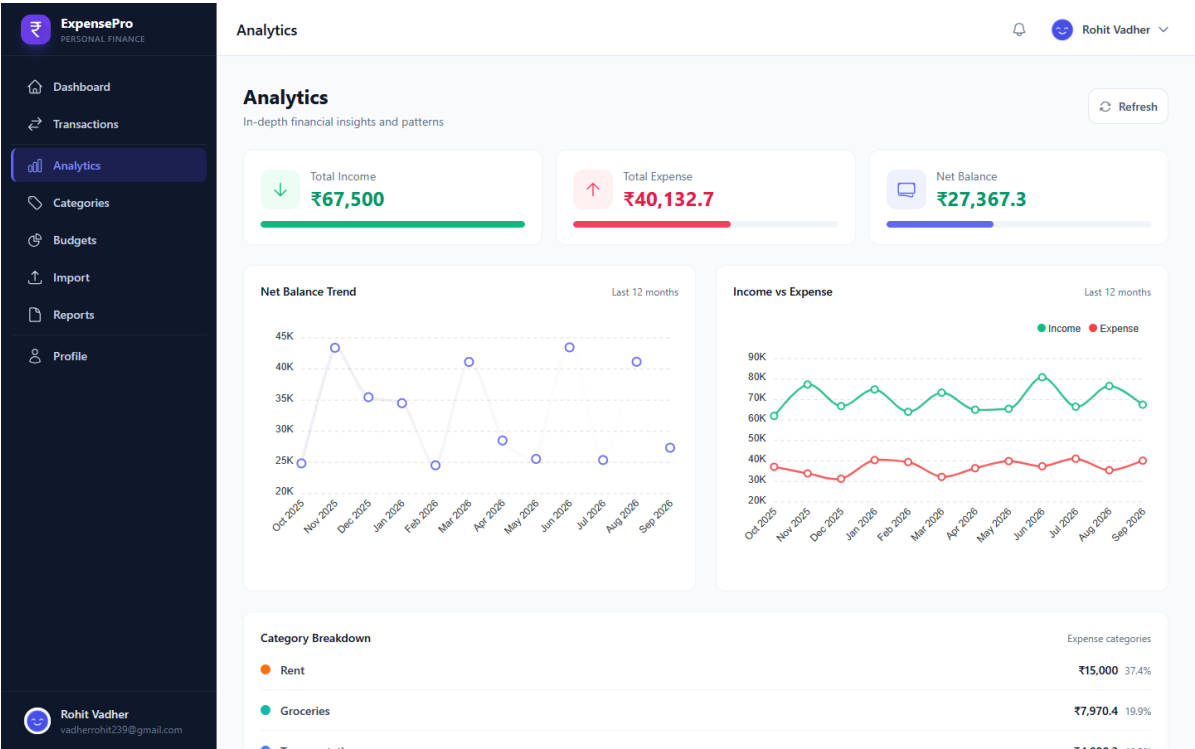


Figure 16: Figure — ExpensePro Analytics with twelve-month charts.

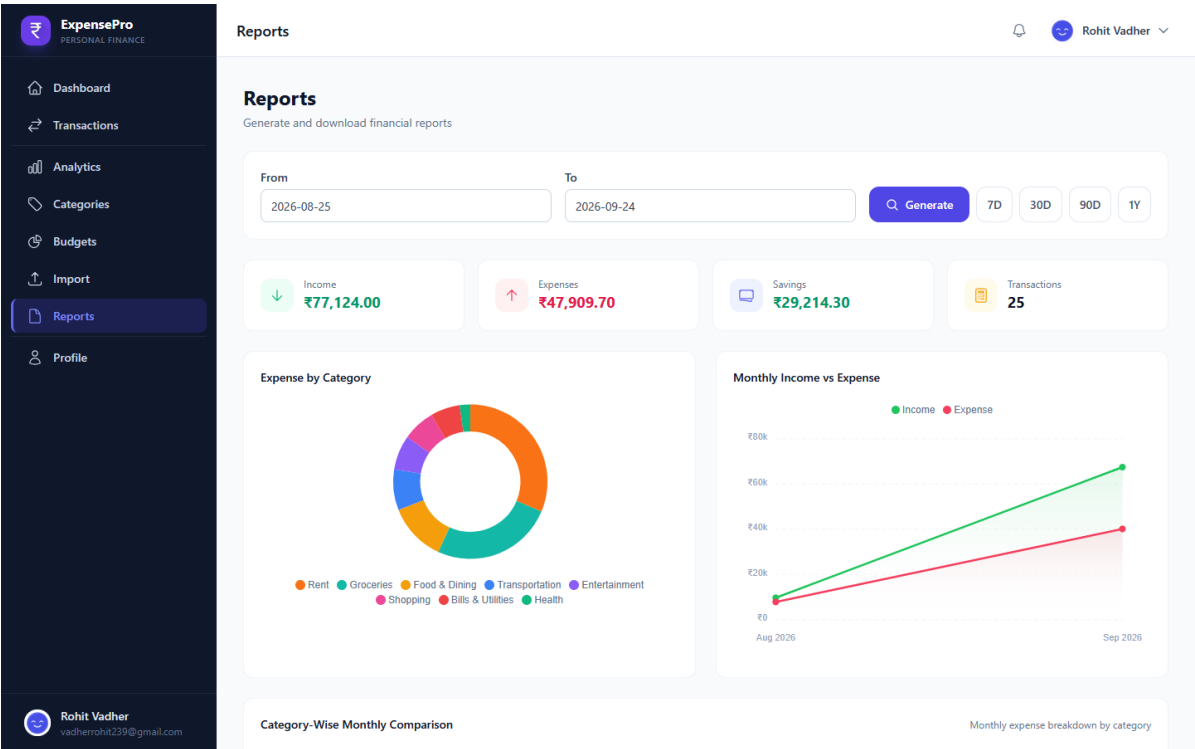


Figure 17: Figure — ExpensePro Reports with exports.

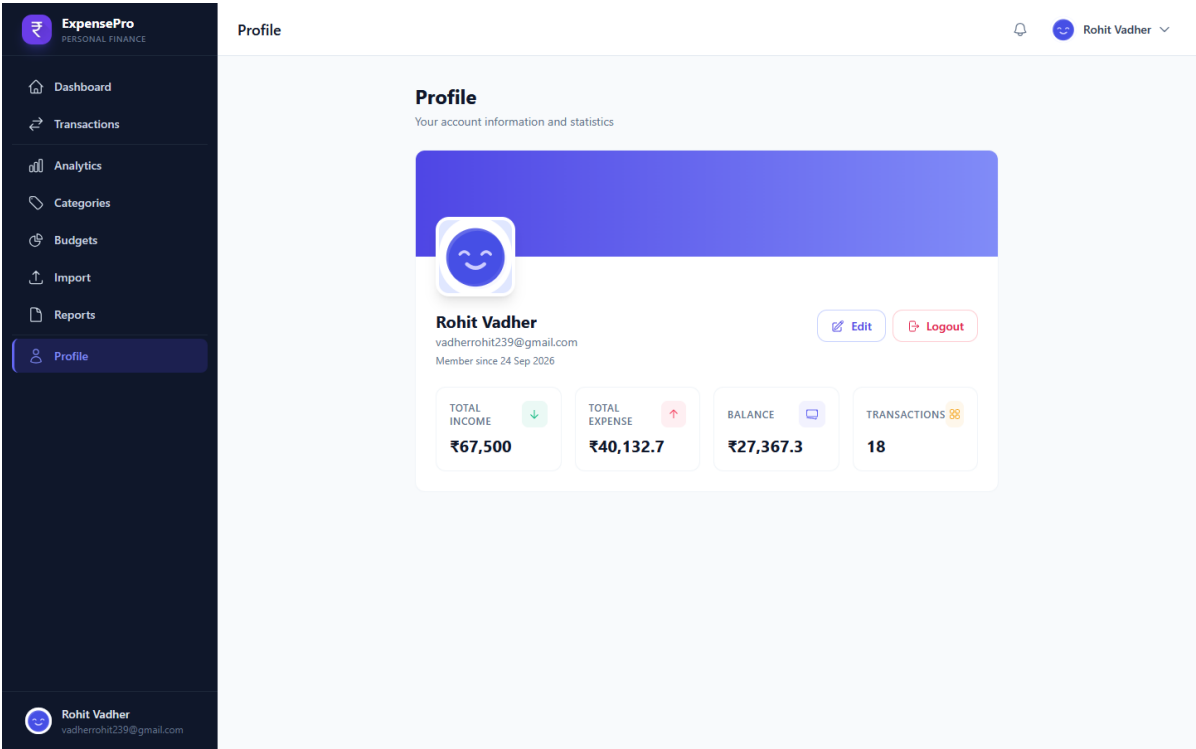


Figure 18: Figure — ExpensePro Profile and stats.

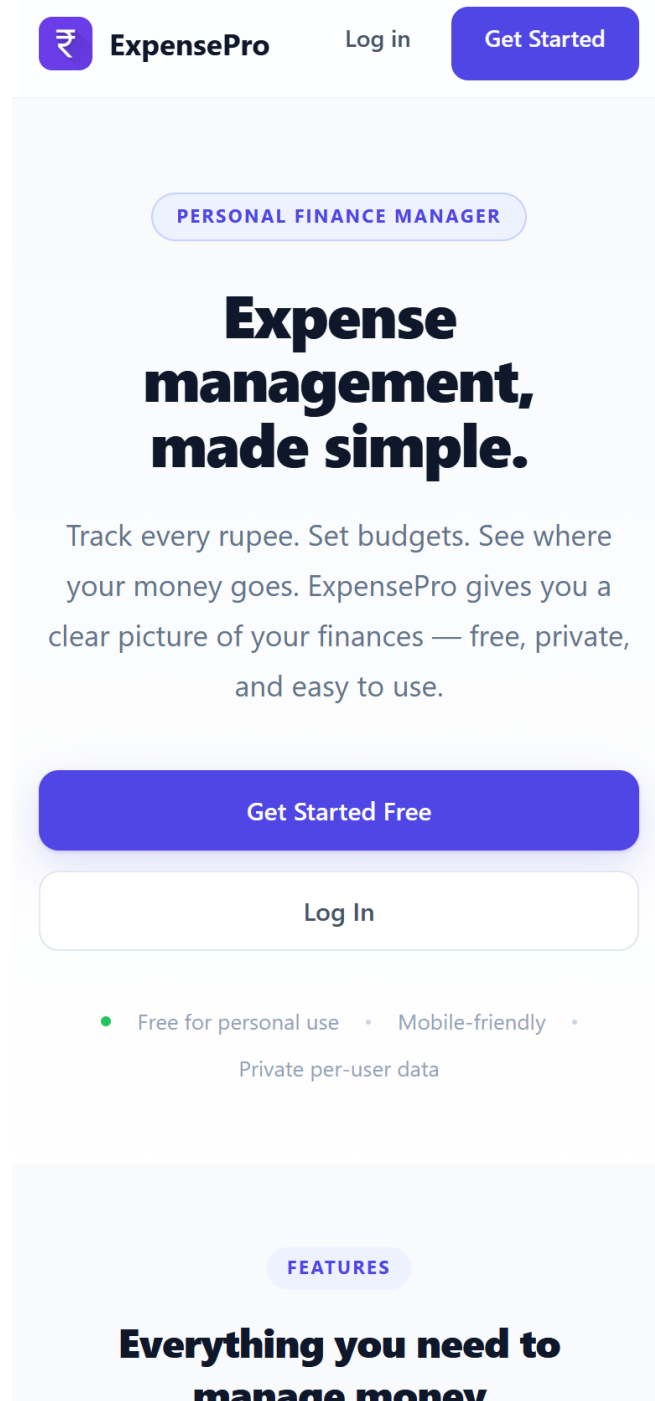
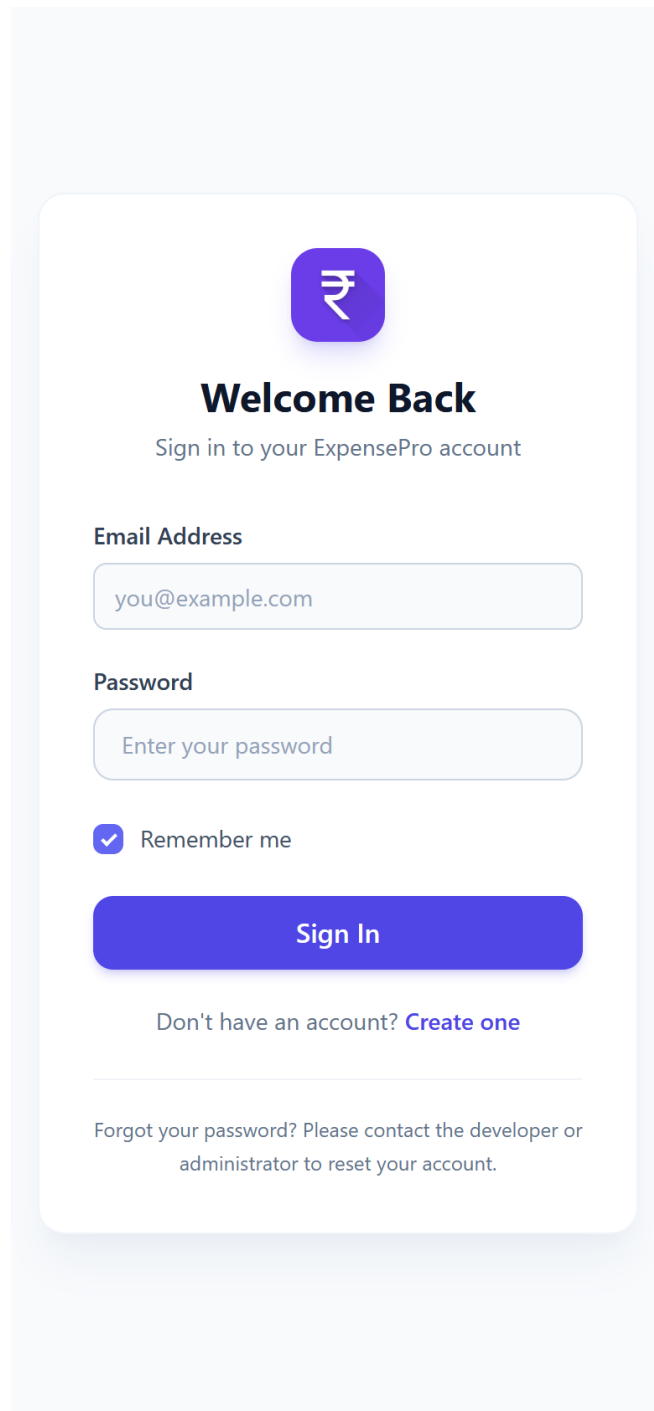


Figure 19: Figure — ExpensePro Mobile landing.



The image shows a mobile login screen for ExpensePro. At the top, there is a purple square icon with a white Indian Rupee symbol (₹). Below the icon, the text "Welcome Back" is displayed in a bold, black font. Underneath, it says "Sign in to your ExpensePro account" in a smaller, regular black font. The form consists of two input fields: "Email Address" with the placeholder text "you@example.com" and "Password" with the placeholder text "Enter your password". Below the password field, there is a checked checkbox followed by the text "Remember me". A large, rounded blue button with the text "Sign In" in white is positioned below the checkbox. Under the button, there is a link that says "Don't have an account? [Create one](#)". At the bottom of the form, there is a line of text: "Forgot your password? Please contact the developer or administrator to reset your account."

Figure 20: Figure — ExpensePro Mobile login.

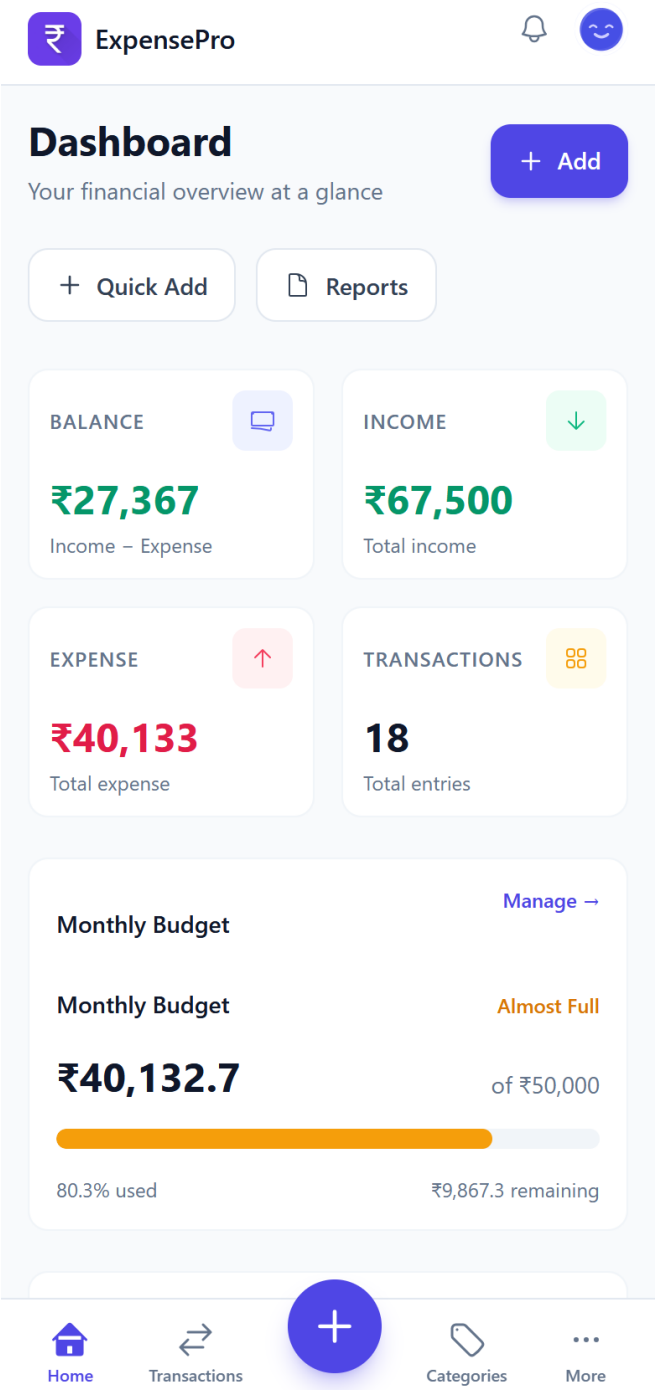


Figure 21: Figure — ExpensePro Mobile dashboard.

Add Transaction

×

Transaction Type

Income

Expense

Category

Select a category

▼

Amount (₹)

0.00

Description (optional)

What was this for?

Date

2026-09-24

Add Transaction

Figure 22: Figure — ExpensePro Mobile transaction bottom sheet.

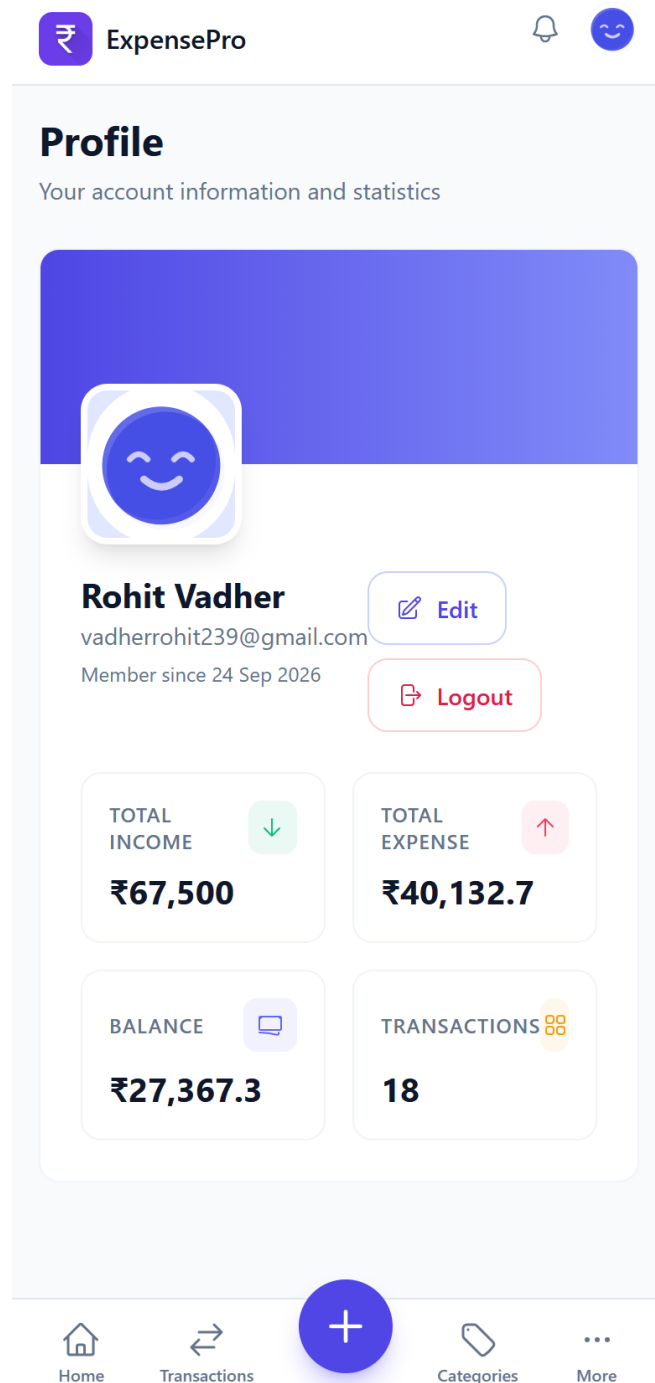


Figure 23: Figure — ExpensePro Mobile profile.

Project Structure

See Folder Architecture. Responsibilities: api (HTTP boundary), pages/layouts (shells), assets/js (core, services, components, features, pages), assets/css (tokens through pages), backend (rules, services, repositories), includes (bootstrap), database (schema + migrations), docs (this pack), offline (offline page).

Limitations

PDF history pages use printer-safe Rs. amounts because PDF standard fonts lack the rupee glyph; very large category sets render in the cover canvas; quick templates are API-complete; charts and PDF engine need their CDN on first use.

Maintenance Notes

Logs are disabled by design (no runtime files are created); errors surface as safe UI messages with correct HTTP codes. Add tables via schema.sql guards plus a migration file; keep every new query user-scoped and prepared; mirror validation rules on both sides.

Conclusion

ExpensePro delivers a complete private finance workflow on ordinary shared-friendly hosting: PHP, PDO and MySQL on the server, vanilla JavaScript in the browser, one validation and calculation model everywhere, and installable offline-capable clients.

Student/Project Information

Rohit Vadher, BCA Sem-5. ExpensePro v1.0, 2026. Stack: PHP 8+, PDO, MySQL (InnoDB, utf8mb4), vanilla JavaScript, Tailwind CSS, Apache/XAMPP.